

VAPT SANITIZER

v1.0.0

Technical User Guide

Local-first security gateway for authorized VAPT -> AI workflows

LOCAL-FIRST | ENCRYPTED MAPPING | PASS / REVIEW / BLOCKED

Stable release - validated baseline

181/181 tests OK | Auto-detection 10/10 PASS | Burp BUILD SUCCESSFUL

SHA-256 stable ZIP

2ee9928dddb789f1f5fda3479ad86e0c0540deabbe3bf97c5ed82cb666479c84

Contents

1. Overview and objectives
2. Security model
3. Architecture
4. Supported profiles and auto-detection
5. Installation and validation
6. CLI usage
7. AI Handoff and Security Gate
- 8. Engagement Mapping Vault**
9. Burp Suite integration
10. Recommended operational workflows
11. Policies and protection modes
- 12. Troubleshooting**
13. Operational security and retention
14. Release status and validation
- A. Appendix - Quick commands
- B. Appendix - New assessment checklist

Audience: Penetration testers, VAPT engineers, Red Team operators, and technical colleagues who need to use AI systems without directly transferring real client identifiers and secrets.

Scope: This guide describes the validated v1.0.0 Stable release. It does not replace NDAs, rules of engagement, data-handling policies, or client contractual requirements.

1. Overview and objectives

VAPT Sanitizer is a local-first gateway designed to place a security boundary between the technical output of an authorized assessment and an Artificial Intelligence system used as an analytical assistant.

The goal is not to indiscriminately hide all content. The value of the tool is to preserve ports, services, versions, paths, evidence, CVEs, and technical relationships while protecting identifiers and secrets that must not be transferred outside the operator workstation.

1.1 The operational problem

```
Nmap scan report for srv-app.client.local (10.20.30.45)
Authorization: Bearer eyJ...
user=m.rossi email=mario.rossi@client.example
AWS_SECRET_ACCESS_KEY=...
```

An LLM can help interpret this data, but the original input may contain information that must not be shared. VAPT Sanitizer transforms the dataset while preserving the context required for analysis.

```
srv-app.client.local -> [HOSTNAME_001]
10.20.30.45         -> [PRIVATE_IP_001]
mario.rossi@...     -> [EMAIL_001]
password=...        -> password=[PASSWORD_REDACTED]
```

1.2 What v1.0 provides

- Local-first sanitization of files, stdin, and clipboard content.
- Automatic profile detection or explicit profile selection.
- Stable pseudonymization of correlatable values.
- Irreversible redaction of mandatory secrets.
- Security Gate PASS / REVIEW / BLOCKED.
- Provider-neutral AI-ready prompts with no automatic transmission.
- Encrypted, persistent Engagement Mapping Vault.
- CLI <-> Burp and cross-tool correlation.
- Burp Suite extension with preview, policy, engagement, and AI Handoff support.
- Synthetic fixtures, regression tests, and release checks.

Operational principle: AI is the analytical assistant; VAPT Sanitizer is the security boundary; the penetration tester remains the final decision maker.

2. Security model

2.1 Pseudonymization

Values that must remain correlatable are replaced with readable placeholders. When the Engagement Mapping Vault is active, the same real value retains the same placeholder across files, processes, and different tools within the same engagement.

```
10.20.30.45 -> [PRIVATE_IP_001]
srv-app.internal.local -> [HOSTNAME_001]
user@client.example -> [EMAIL_001]
```

2.2 Redaction

Secrets must not be recoverable. They are therefore redacted and are never stored in the Mapping Vault.

```
password=SuperSecret -> password=[PASSWORD_REDACTED]
Authorization: Bearer eyJ... -> Authorization: [BEARER_TOKEN_REDACTED]
PRIVATE KEY block -> [PRIVATE_KEY_REDACTED]
```

2.3 Mandatory secret protections

- PASSWORD
- BEARER_TOKEN
- JWT
- PRIVATE_KEY
- AWS_ACCESS_KEY
- AWS_SECRET_KEY
- AWS_SESSION_TOKEN
- API_KEY
- SESSION
- AUTH_CREDENTIAL
- DATABASE_PASSWORD
- PROVIDER_TOKEN
- GENERIC_SECRET

Invariant: A YAML policy cannot turn a mandatory secret into freely exportable data. Secret redaction remains a barrier that policy files cannot weaken.

2.4 Trust boundary

The vault encrypts mappings at rest and keeps the key separate from the ciphertext. This reduces accidental exposure and protects backups and repositories, but it does not defend against an attacker who has already compromised the local account and can read both master.key and mapping.enc.

3. Architecture

INPUT Nmap / Burp / Gobuster / Nuclei / Windows / Linux / AWS / file / clipboard
CONTEXT Profile detection and tool metadata
DETECTION Detectors + policy engine
TRANSFORM Pseudonymization / redaction
VAULT Encrypted local mapping per engagement - pseudonymized values only
GATE PASS / REVIEW / BLOCKED
HANDOFF Provider-neutral AI-ready prompt
CLIPBOARD Verified local copy

OPERATOR
Manual paste into the AI browser and local placeholder resolution

No automatic transmission: In the standard AI Handoff workflow, prompt generation performs no network requests. Transfer to the AI service remains a manual operator action.

4. Supported profiles and auto-detection

Profile	Typical input	Main behavior
GENERIC	Logs and mixed content	Hostname, IP, username, email, secrets; preserves general technical context
NMAP	Output Nmap	Target host/IP/MAC; preserves ports, services, versions, CVEs, and NSE output
BURP	HTTP request/response	Host, identity, auth headers, sessions, secrets; preserves HTTP structure
GOBUSTER	Directory/file enumeration	Target metadata; preserves paths, files, status codes, and wordlists
NUCLEI	Nuclei findings	Target identifiers; preserves templates, severity, CVEs, and public references
WINDOWS	Windows/AD enumeration	Hostname, SID, domain, IP; preserves standard Windows context
LINUX	Linux enumeration/log	Hostname, environment users, IDs, IP/IPv6/MAC; preserves standard service users
AWS	Cloud/AWS output	Account/resource/IAM identifiers; redacts AWS credentials and tokens
SECRETS	Secret-heavy data	Password, token, API key, private key, DSN, credential material
PII	Personal data	Name, phone, DOB, tax/fiscal ID, address, city, postal code, IBAN, username

4.1 Auto-detection

If `--profile` is not specified, the program uses automatic profile detection. The release includes dedicated synthetic fixtures, and the official validation matrix is 10/10 PASS.

```
./venv/bin/vapt-sanitize scan.txt -o scan.sanitized
# profile selected automatically
```

4.2 When to force a profile

Use an explicit profile when the input is ambiguous, contains multiple non-standard formats, or deterministic behavior is required for a documented procedure.

```
./venv/bin/vapt-sanitize scan.txt --profile nmap
```

5. Installation and validation

5.1 Requirements

Item	Details
Python	3.11 or later with venv support
Python dependencies	PyYAML 6.0.3; cryptography 46.0.4
Clipboard Linux	xclip / xsel / wl-copy optional
Burp build	Java 17+, unzip, curl or wget
Gradle	Not required system-wide; Gradle 8.14.3 bootstrap included

5.2 Stable installation

Directory-name note: The Stable ZIP is byte-for-byte identical to the validated RC2, so it still extracts to vapt-sanitize-v1.0.0-rc2/. The application itself still reports version 1.0.0.

```
unzip vapt-sanitize-v1.0.0.zip
cd vapt-sanitize-v1.0.0-rc2
./scripts/install.sh
```

The script creates .venv, installs the project in editable mode with pinned dependencies, checks the clipboard backend, and runs the regression suite.

5.3 Verification

```
./venv/bin/python -m vapt_sanitize --version
./scripts/release-check.sh
```

```
VAPT Sanitizer 1.0.0
181 tests OK
Auto-detection result: 10/10 PASS
[+] Release checks passed.
```

5.4 Burp operational directory

The Burp UI defaults to ~/vapt-sanitize and ~/vapt-sanitize/.venv/bin/python. For a versioned installation, use an operational symlink/rename or launch Burp with explicit environment variables.

```
export VAPT_SANITIZE_HOME="$HOME/vapt-sanitize-v1.0.0-rc2"
export VAPT_SANITIZE_PYTHON="$VAPT_SANITIZE_HOME/.venv/bin/python"
burpsuite
```

6. CLI usage

6.1 Basic commands

```
# File -> sanitized output
./venv/bin/vapt-sanitize scan.txt -o scan.sanitized

# Stdin
cat scan.txt | ./venv/bin/vapt-sanitize -

# Explicit profile
./venv/bin/vapt-sanitize scan.txt --profile nmap

# Explain decisions
./venv/bin/vapt-sanitize scan.txt --explain
```

6.2 Clipboard

```
# Read the clipboard and replace it with sanitized content
./venv/bin/vapt-sanitize --clipboard

# Read the clipboard and replace it with the AI-ready prompt
./venv/bin/vapt-sanitize --clipboard --ai-prompt
```

6.3 AI prompt from file or stdin

```
./venv/bin/vapt-sanitize scan.txt \
  --ai-prompt \
  --ai-clipboard
```

6.4 Available AI tasks

- analyze-security
- explain-response
- find-attack-surface
- suggest-next-tests
- custom

```
./venv/bin/vapt-sanitize burp.txt \
  --ai-prompt --ai-clipboard \
  --task custom \
  --question "Which authorization tests should I prioritize?"
```

Source label: The prompt Source field is derived from the profile/tool; it can be overridden with --source without automatically including the engagement ID.

7. AI Handoff and Security Gate

The Security Gate is evaluated before the AI-ready prompt is generated. Its purpose is to prevent the handoff from becoming an automatic "copy everything" operation.

State	Meaning	Behavior
PASS	No implemented rule requires operator intervention	Handoff allowed
REVIEW	Manual inspection and approval required	CLI: --review-approved; Burp: confirmation dialog
BLOCKED	The boundary conditions are not satisfied	Handoff refused; review approval cannot bypass it

```
./venv/bin/vapt-sanitize input.txt \
  --ai-prompt --ai-clipboard \
  --review-approved
```

Correct interpretation: PASS is not a mathematical guarantee of universal anonymization. It means the implemented controls did not detect conditions requiring REVIEW or BLOCKED. Manual preview remains an operational requirement.

7.1 Prompt format

```
You are assisting with an authorized security assessment.
Source: Nmap enumeration

Analyze this sanitized security-testing data...
Do not attempt to reconstruct, infer, or guess original values.

--- SANITIZED DATA ---
Nmap scan report for [HOSTNAME_001] ([PRIVATE_IP_001])
```

8. Engagement Mapping Vault

The Mapping Vault solves a key problem in multi-tool workflows: without persistent state, two separate processes could assign the same placeholder to different real values or different placeholders to the same host. With an engagement, mappings are persistent and isolated per assessment.

8.1 Creation

```
./venv/bin/vapt-sanitize engagement init ACME-2026-EXTERNAL
```

The ID must be 1-64 characters long, start with a letter or number, and may contain letters, numbers, dots, underscores, and hyphens.

8.2 Cross-tool usage

```
./venv/bin/vapt-sanitize scan.txt \
  --engagement ACME-2026-EXTERNAL \
  --ai-prompt --ai-clipboard
```

```
Nmap: 10.20.30.45 -> [PRIVATE_IP_001]
Burp: 10.20.30.45 -> [PRIVATE_IP_001]
Nuclei: 10.20.30.45 -> [PRIVATE_IP_001]
```

8.3 Show and resolve

```
./venv/bin/vapt-sanitize engagement show ACME-2026-EXTERNAL

./venv/bin/vapt-sanitize engagement resolve \
ACME-2026-EXTERNAL \
"[PRIVATE_IP_001]" "[HOSTNAME_001]"
```

8.4 Paths and protections

```
~/.config/vapt-sanitize/master.key
~/.local/share/vapt-sanitize/engagements/<ID>/mapping.enc
~/.local/share/vapt-sanitize/engagements/<ID>/metadata.json
~/.local/share/vapt-sanitize/engagements/<ID>/mapping.lock
```

- The key and ciphertext are stored separately.
- The mapping is encrypted at rest with authenticated encryption through cryptography/Fernet.
- Directories and files are created with restrictive permissions when supported by the operating system.
- The per-engagement lock serializes concurrent updates.
- Redacted secrets are never stored in the vault.
- Losing master.key makes existing mappings undecryptable.

Backup: If correlation must survive reinstallation or workstation replacement, keep a protected offline backup of master.key together with an approved retention strategy.

9. Burp Suite integration

9.1 Build

```
./scripts/build-burp.sh
```

The included bootstrap uses Gradle 8.14.3 with SHA-256 verification. Installing Gradle from the Kali repositories is not required.

```
burp-extension/build/libs/vapt-sanitize-burp-1.0.0.jar
```

9.2 Installing in Burp

1. Open Burp Suite.
2. Extensions -> Installed -> Add.
3. Choose Java as the extension type.
4. Select vapt-sanitize-burp-1.0.0.jar.
5. Open the VAPT Sanitizer tab and verify the runtime status.

9.3 Engagement panel

```
Engagement Mapping Vault
Active engagement: [ (None - stateless) v ] [ Refresh ]
```

Every new session intentionally starts in Stateless mode. This reduces the risk of contaminating a new project with the previously selected client. Click Refresh to reload local vaults and select the correct engagement.

9.4 Context menu and preview

```
VAPT Sanitizer
Engagement: ACME-2026-EXTERNAL
Sanitize request -> preview
Sanitize response -> preview
Sanitize selection -> preview
Policy
```

The preview shows the Security Gate, input type, policy, findings, engagement, detections, and sanitized content. The main actions are Copy sanitized and Copy AI Prompt. REVIEW requires confirmation; BLOCKED disables the action.

9.5 CLI <-> Burp correlation

```
CLI/Nmap: srv-app.internal.local -> [HOSTNAME_001]
Burp: Host: srv-app.internal.local -> Host: [HOSTNAME_001]
```

9.6 Runtime path

```
export VAPT_SANITIZE_HOME="$HOME/vapt-sanitize-v1.0.0-rc2"
export VAPT_SANITIZE_PYTHON="$VAPT_SANITIZE_HOME/.venv/bin/python"
burpsuite
```

10. Recommended operational workflows

10.1 Standard assessment workflow

1. Create an engagement aligned with the client and scope.
2. Run the authorized VAPT tools normally.
3. Save or capture the outputs.
4. Always sanitize using the same engagement.
5. Check profile detection, findings, and Security Gate state.
6. Generate the AI-ready prompt.
7. Visually review the prompt.
8. Manually paste the prompt into the chosen AI service.
9. Read the analysis using placeholders as persistent identities.
10. Use engagement resolve to return to real values when needed.
11. Continue the assessment against the real environment and document the evidence.

10.2 Engagement naming convention

```
CLIENT-YEAR-SCOPE

ACME-2026-EXTERNAL
ACME-2026-WEBAPP
ACME-2026-INTERNAL
```

Fundamental rule: Do not reuse the same engagement across different clients. Do not use different engagements for files from the same assessment if placeholder correlation must be preserved.

10.3 When to use Stateless mode

- Isolated synthetic fixtures and tests.
- A single output with no need for correlation.
- Quick checks where persistent mapping is unnecessary.

11. Policies and protection modes

Item	Details
default.yml	Balanced: pseudonymizes correlatable identifiers and redacts secrets/credit-card data.
aggressive_pii.yml	Redacts more personal data, including several PII fields and usernames across more profiles.
strict_llm.yml	Tightens LLM handoff blocking: preserved findings and no-findings states become BLOCKED; redacts email/username in relevant profiles.

```
./venv/bin/vapt-sanitize input.txt \
--policy policies/strict_llm.yml \
--ai-prompt
```

Policies modify the handling of non-mandatory categories and gate logic, but they cannot disable redaction of mandatory secrets.

12. Troubleshooting

Engagement vaults require the cryptography package

The process using the vault is not running the correct virtual environment. Verify `./venv/bin/python` and reinstall with `./scripts/install.sh` or `./venv/bin/python -m pip install -e .`

Burp does not show the engagement

Create it from the CLI, click Refresh, and verify that Burp and the CLI use the same XDG config/data directories or `VAPT_SANITIZE_*` overrides.

CLI and Burp assign different placeholders

Check the engagement ID, master.key, and data root. Both processes must share the same vault.

Clipboard mode fails

Install `xclip/xsel/wl-copy` and ensure the graphical session exposes the correct clipboard.

Security Gate = REVIEW

Inspect the content; if approved, use `--review-approved` or confirm the Burp review dialog.

Security Gate = BLOCKED

Do not bypass it. Correct the input, policy, or triggering condition and sanitize again.

Burp build: Gradle not found

Use the Stable release and `./scripts/build-burp.sh`. The Gradle 8.14.3 bootstrap is included; Java 17+, unzip, and curl/wget are required.

Master key not found / invalid

Do not blindly regenerate the key if vaults must be preserved. Restore the correct master.key backup; a different key cannot decrypt mapping.enc.

```
./venv/bin/python -c "from cryptography.fernet import Fernet; print('cryptography OK')"
```

13. Operational security and retention

13.1 Files to protect

```
~/config/vapt-sanitize/master.key  
~/local/share/vapt-sanitize/engagements/  
client outputs / sanitized reports / correlatable notes
```

- Do not commit keys or mappings to Git.
- Do not attach the vault to tickets or external chats.
- Do not include mappings or real values in AI prompts.
- Apply the retention period required by contract, NDA, or internal policy.
- Treat mapping data as client material even when encrypted.
- Protect the workstation: the operating system is part of the trust boundary.

13.2 Limitations

- The tool is not a universal DLP system.
- New formats may require future detectors or profiles.
- False positives and false negatives are possible.
- Human preview remains part of the process.
- VAPT Sanitizer does not replace vulnerability scanners, exploit frameworks, SIEMs, or password managers.

14. Release status and validation

Item	Details
Version	VAPT Sanitizer 1.0.0 Stable
Regression suite	181/181 tests OK
Auto-detection fixtures	10/10 PASS
Burp build	BUILD SUCCESSFUL
JAR	vapt-sanitize-burp-1.0.0.jar
Python	>= 3.11
cryptography	46.0.4
PyYAML	6.0.3
Java	>= 17
Gradle bootstrap	8.14.3
Montoya API	2026.7

14.1 Stable artifact

vapt-sanitize-v1.0.0.zip
SHA-256:
2ee9928dddb789f1f5fda3479ad86e0c0540deabbe3bf97c5ed82cb666479c84

Golden baseline: Stable was promoted byte-for-byte from RC2 after it passed 181 tests, 10/10 auto-detection, and BUILD SUCCESSFUL. New functionality should start from a subsequent version without altering the v1.0.0 artifact.

Appendix A - Quick commands

```
# Version
./venv/bin/vapt-sanitize --version

# Sanitize file
./venv/bin/vapt-sanitize input.txt -o output.sanitized

# AI prompt -> clipboard
./venv/bin/vapt-sanitize input.txt --ai-prompt --ai-clipboard

# Create engagement
./venv/bin/vapt-sanitize engagement init CLIENT-2026-001

# Use engagement
./venv/bin/vapt-sanitize input.txt --engagement CLIENT-2026-001 --ai-prompt --ai-clipboard

# Show mapping
./venv/bin/vapt-sanitize engagement show CLIENT-2026-001

# Resolve placeholder
./venv/bin/vapt-sanitize engagement resolve CLIENT-2026-001 "[HOSTNAME_001]"

# Release check
./scripts/release-check.sh

# Build Burp
./scripts/build-burp.sh
```

Appendix B - New assessment checklist

- ☐ Engagement ID created and aligned with the client/scope.
- ☐ master.key protected and, if required, an offline backup is available.
- ☐ Rules of engagement authorize the planned tests.
- ☐ Policy selected deliberately.
- ☐ Burp launched with the correct VAPT_SANITIZE_HOME/PYTHON values for a versioned installation.
- ☐ Burp Engagement Mapping Vault set to the correct client, not Stateless.
- ☐ Every AI Handoff visually reviewed.
- ☐ REVIEW approved only after inspection; BLOCKED never bypassed.
- ☐ Mapping data never pasted into AI.
- ☐ Vault retention defined for the end of the assessment.

Final principle: Preserve the technical context useful for AI analysis without turning the AI service into a repository of real client data. Correlation remains local; judgment remains human.